Final Project Report: LLM-Based Hardware Design and Optimization

1. Challenges and Difficulties Encountered

Throughout the project, several technical hurdles were identified during the transition from LLM-generated code to a verified hardware design:

- **Syntax and Indexing Errors**: Initial LLM outputs frequently contained indexing mistakes (e.g., c[7:1] = g3[6:0] missing the final carry or incorrect loop boundaries).
- **Interface Mismatch**: The LLM often added unnecessary ports like cin which caused formal verification (Yosys) to fail when compared against golden models that lacked that port.
- **Toolchain Compatibility**: Significant issues arose with Yosys script execution on Windows, where invisible carriage return characters (\r) led to "Invalid number of arguments" errors, requiring manual command-line intervention.
- **Sub-module Redundancy**: Concurrent loading of files with duplicate module definitions (like FA) caused namespace collisions during equivalence checking.

---

2. Process Summary

The design flow followed a structured "Human-in-the-Loop" methodology:

1. **Generation**: Used natural language prompts to generate initial 8-bit Ripple Carry (RCA) and Kogge-Stone (KSA) adders.
2. **Simulation**: Verified functional correctness using Icarus Verilog and GTKWave.
3. **Human Intervention**: Manually debugged the KSA logic to ensure the parallel prefix stages (Span 1, 2, 4) were correctly mapped.
4. **Formal Verification**: Used Yosys to prove logical equivalence between the optimized KSA and the golden RCA model.

---

3. Optimization Results: RCA vs. KSA

The transition from a basic RCA to an optimized KSA demonstrated significant PPA (Power, Performance, Area) trade-offs:

Architecture	Logic Depth	Delay Scaling	Result
Ripple Carry (RCA)	8 Stages	O(N) - Linear	High latency, small area .
Kogge-Stone (KSA)	3 Stages	O(\log_2 N) - Logarithmic	**Selected**: 62.5% reduction in logic depth for high-speed performance .

---

4. LLM Analysis for Hardware Design

A. Your Experience with LLM-based Hardware Design

Using an LLM accelerates the "boilerplate" phase of design, such as generating module headers and repetitive assignments. However, the experience highlighted that the LLM acts more as a **coprocessor** than a standalone designer; it requires an engineer to guide the architecture and verify the mathematical boundary conditions.

B. Accuracy and Limitations of LLM-generated Code

- **Accuracy**: High for standard templates (RCA) but lower for complex, wire-heavy architectures (KSA).
- **Limitations**: LLMs often "hallucinate" port lists or fail to maintain consistency across large bit-widths. They struggle with the strictness of hardware description languages (HDL) where a single off-by-one index error renders the entire circuit useless.

C. Quality of LLM-generated Testbenches

- **Strengths**: Excellent at generating basic stimulus (clocks, resets, and simple exhaustive loops).
- **Weaknesses**: Often lacks comprehensive coverage for corner cases (e.g., maximum overflow or specific carry-chain triggers). Testbenches may also miss self-checking logic, requiring manual assert or if-else verification blocks.

D. Lessons Learned and Best Practices

- **Modular Prompting**: Ask the LLM to generate small, verifiable sub-modules (like BigCircle or Square) rather than an entire complex system at once.
- **Strict Interface Control**: Explicitly define port lists in the prompt to prevent the LLM from adding or removing signals like cin.

- **Format Cleanliness**: Always ensure scripts are saved with Unix (LF) line endings to avoid toolchain parsing errors.

E. Recommendations for using LLMs in Hardware Design

1. **Iterative Refinement**: Use the LLM to generate a draft, then use simulation errors to "feedback" the error logs to the LLM for correction.
2. **Formal Verification is Mandatory**: Because LLMs can produce logic that "looks" correct but has subtle bugs, tools like Yosys are essential to prove equivalence.
3. **Hybrid Approach**: Let the LLM handle the structural connectivity while the human engineer defines the critical timing paths and architectural constraints.